

DSA LAB FILE-2

NAME- SANCHI GOYAL

BATCH- 52

SAP ID- 590027285

1. Stack

1.Implementation of Stack Data Structure using Arrays in C

```
#include <stdio.h>
#define MAX 10

int stack[MAX];
int top = -1;

void push(int value) {
    if (top == MAX - 1) {
        printf("Stack overflow!\n");
        return;
    }
    stack[++top] = value;
}

int pop() {
    if (top == -1) {
        printf("Stack underflow!\n");
        return -1;
    }
    return stack[top--];
}

int peek() {
    return stack[top];
}

int main() {
    push(10);
    push(20);
    push(30);
    push(40);

    printf("Top: %d\n", peek());
    printf("Pop: %d\n", pop());
    printf("Pop: %d\n", pop());
    return 0;
}
```

2. Stack using Linked List (Dynamic)

```
#include <stdio.h>
#include <stdlib.h>

// creating a node
struct node {
    int data;
    struct node* next;
};

struct node* top = NULL; // top of stack

// push operation
void push(int x) {
    struct node* newnode;
    newnode = (struct node*)malloc(sizeof(struct node));

    if (newnode == NULL) {
        printf("Stack Overflow\n");
    } else {
        newnode->data = x;
        newnode->next = top;
        top = newnode;
        printf("%d inserted\n", x);
    }
}

// pop operation
void pop() {
    struct node* temp;

    if (top == NULL) {
        printf("Stack Underflow\n");
    } else {
        temp = top;
        printf("%d deleted\n", top->data);
        top = top->next;
        free(temp);
    }
}

// display stack
void display() {
    struct node* temp;

    if (top == NULL) {
        printf("Stack is empty\n");
    } else {
        temp = top;
        printf("Stack elements are:\n");
        while (temp != NULL) {
            printf("%d\n", temp->data);
        }
    }
}
```

```

        temp = temp->next;
    }
}

int main() {
    int ch, val;

    while (1) {
        printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &ch);

        switch (ch) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                push(val);
                break;

            case 2:
                pop();
                break;

            case 3:
                display();
                break;

            case 4:
                exit(0);

            default:
                printf("Wrong choice\n");
        }
    }

    return 0;
}

```

2. Queue

3.Implementation of queue using array

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

int main() {
    int val;

    // ENQUEUE (Insert elements)
    printf("Inserting elements:\n");

    val = 10;
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
    } else {
        if (front == -1)
            front = 0;
        queue[++rear] = val;
        printf("%d inserted\n", val);
    }

    val = 20;
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
    } else {
        if (front == -1)
            front = 0;
        queue[++rear] = val;
        printf("%d inserted\n", val);
    }

    val = 30;
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
    } else {
        if (front == -1)
            front = 0;
        queue[++rear] = val;
        printf("%d inserted\n", val);
    }

    // DISPLAY
    printf("\nQueue elements are:\n");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
}
```

```

// DEQUEUE (Delete element)
printf("\n\nDeleting element:\n");
if (front == -1 || front > rear) {
    printf("Queue Underflow\n");
} else {
    printf("%d deleted\n", queue[front]);
    front++;
}

// DISPLAY again
printf("\nQueue after deletion:\n");
for (int i = front; i <= rear; i++) {
    printf("%d ", queue[i]);
}

return 0;
}

```

4. Implementation of queue using a linked list

```

#include <stdio.h>
#include <stdlib.h>

// node structure
struct node {
    int data;
    struct node* next;
};

int main() {
    struct node *front = NULL, *rear = NULL, *newnode, *temp;
    int val;

    // ENQUEUE (insert elements)
    printf("Inserting elements:\n");

    val = 10;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = NULL;

    if (front == NULL) {
        front = rear = newnode;
    } else {
        rear->next = newnode;
        rear = newnode;
    }
    printf("%d inserted\n", val);
}

```

```
val = 20;
newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = val;
newnode->next = NULL;
```

```
rear->next = newnode;
rear = newnode;
printf("%d inserted\n", val);
```

```
val = 30;
newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = val;
newnode->next = NULL;
```

```
rear->next = newnode;
rear = newnode;
printf("%d inserted\n", val);
```

```
// DISPLAY
printf("\nQueue elements are:\n");
temp = front;
while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}
```

```
// DEQUEUE (delete element)
printf("\n\nDeleting element:\n");
if (front == NULL) {
    printf("Queue Underflow\n");
} else {
    temp = front;
    printf("%d deleted\n", front->data);
    front = front->next;
    free(temp);
}
```

```
// DISPLAY again
printf("\nQueue after deletion:\n");
temp = front;
while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}
```

```
return 0;
```

```
}
```

5. Implementation of Circular Queue Using Array

```
#include <stdio.h>
#define MAX 5

int cq[MAX];
int front = -1, rear = -1;

// (Enqueue)
void NQ(int x) {
    if ((rear + 1) % MAX == front) {
        printf("Overflow\n");
        return;
    }

    if (front == -1)
        front = 0;

    rear = (rear + 1) % MAX;
    cq[rear] = x;
}

// (Dequeue)
void DQ() {
    if (front == -1) {
        printf("Underflow\n");
        return;
    }

    printf("%d deleted\n", cq[front]);

    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

// Display
void display() {
    if (front == -1) {
        printf("Queue empty\n");
        return;
    }

    int i = front;
    while (1) {
        printf("%d ", cq[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}
```



```

int main() {
    // inserting
    NQ(10);
    NQ(20);
    NQ(30);
    NQ(40);

    printf("Queue: ");
    display();

    // deleting
    DQ();

    printf("After deletion: ");
    display();

    return 0;
}

```

6.Implementation of Circular Queue using Link List

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

struct node *front = NULL, *rear = NULL;

// NQ (Enqueue)
void NQ(int x) {
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = x;

    if (front == NULL) {
        front = rear = newnode;
        rear->next = front; // circular link
    } else {
        rear->next = newnode;
        rear = newnode;
        rear->next = front; // maintain circular
    }
}

// DQ (Dequeue)
void DQ() {
    struct node* temp;

```

```

    if (front == NULL) {
        printf("Underflow\n");
        return;
    }

    temp = front;
    printf("%d deleted\n", front->data);

    if (front == rear) {
        front = rear = NULL;
    } else {
        front = front->next;
        rear->next = front; // maintain circular
    }

    free(temp);
}

// Peek (see front element)
void peek() {
    if (front == NULL)
        printf("Queue empty\n");
    else
        printf("Front element is %d\n", front->data);
}

// Display
void display() {
    if (front == NULL) {
        printf("Queue empty\n");
        return;
    }

    struct node* temp = front;

    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != front);

    printf("\n");
}

int main() {
    // inserting
    NQ(10);
    NQ(20);
    NQ(30);

    printf("Queue: ");
    display();
}

```

```

// peek
peek();

// deleting
DQ();

printf("After deletion: ");
display();

return 0;
}

```

7.Implementation of a queue from a stack using an array

```

#include <stdio.h>
#define N 5

int s1[N], s2[N];
int top1 = -1, top2 = -1;

// enqueue operation
void enqueue(int x) {
    if (top1 == N - 1) {
        printf("Overflow\n");
        return;
    }

    s1[++top1] = x;
}

// dequeue operation
void dequeue() {
    if (top1 == -1 && top2 == -1) {
        printf("Underflow\n");
        return;
    }

    // move elements from s1 to s2 if s2 is empty
    if (top2 == -1) {
        while (top1 != -1) {
            s2[++top2] = s1[top1--];
        }
    }

    printf("%d deleted\n", s2[top2--]);
}

// display queue
void display() {
    if (top1 == -1 && top2 == -1) {

```

```

        printf("Queue empty\n");
        return;
    }

    // print s2 (front part)
    for (int i = top2; i >= 0; i--) {
        printf("%d ", s2[i]);
    }

    // print s1 (rear part)
    for (int i = 0; i <= top1; i++) {
        printf("%d ", s1[i]);
    }

    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);

    printf("Queue: ");
    display();

    dequeue();

    printf("After deletion: ");
    display();

    return 0;
}

```

8.Double-ended queue

```

#include <stdio.h>
#define N 5

int dq[N];
int front = -1, rear = -1;

// insert from front
void insertFront(int x) {
    if ((front == 0 && rear == N - 1) || (front == rear + 1)) {
        printf("Overflow\n");
        return;
    }

    if (front == -1) {
        front = rear = 0;
    } else if (front == 0) {

```

```

        front = N - 1;
    } else {
        front--;
    }

    dq[front] = x;
}

// insert from rear
void insertRear(int x) {
    if ((front == 0 && rear == N - 1) || (front == rear + 1)) {
        printf("Overflow\n");
        return;
    }

    if (front == -1) {
        front = rear = 0;
    } else if (rear == N - 1) {
        rear = 0;
    } else {
        rear++;
    }

    dq[rear] = x;
}

// delete from front
void deleteFront() {
    if (front == -1) {
        printf("Underflow\n");
        return;
    }

    printf("%d deleted\n", dq[front]);

    if (front == rear) {
        front = rear = -1;
    } else if (front == N - 1) {
        front = 0;
    } else {
        front++;
    }
}

// delete from rear
void deleteRear() {
    if (front == -1) {
        printf("Underflow\n");
        return;
    }

    printf("%d deleted\n", dq[rear]);
}

```

```

    if (front == rear) {
        front = rear = -1;
    } else if (rear == 0) {
        rear = N - 1;
    } else {
        rear--;
    }
}

// display deque
void display() {
    if (front == -1) {
        printf("Deque empty\n");
        return;
    }

    int i = front;
    while (1) {
        printf("%d ", dq[i]);
        if (i == rear)
            break;
        i = (i + 1) % N;
    }
    printf("\n");
}

int main() {
    insertRear(10);
    insertRear(20);
    insertFront(5);
    insertRear(30);

    printf("Deque: ");
    display();

    deleteFront();
    deleteRear();

    printf("After deletion: ");
    display();

    return 0;
}

```

3.TREE

9.Binary search tree

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} node;

// create node
node* createnode(int data) {
    node* temp = (node*)malloc(sizeof(node));

    if (temp == NULL) {
        printf("Memory allocation failed\n");
        return NULL;
    }

    temp->data = data;
    temp->left = NULL;
    temp->right = NULL;

    return temp;
}

// insert into BST
node* insert(node* root, int data) {
    if (root == NULL)
        return createnode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root;
}

void inorderTraversal(node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

void preorderTraversal(node* root) {
    if (root != NULL) {
```

```

        printf("%d ", root->data);
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}

```

```

void postorderTraversal(node* root) {
    if (root != NULL) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        printf("%d ", root->data);
    }
}

```

// find level of a node

```

int find(node* root, int value, int count) {
    if (root == NULL)
        return -1;

    count++;

    if (value == root->data)
        return count;

    if (value < root->data)
        return find(root->left, value, count);
    else
        return find(root->right, value, count);
}

```

```

int main() {
    node* root = NULL;
    node* root2 = NULL;

```

```

    int values[14] = {90,60,140,40,70,120,170,20,50,10,30,100,130,160};
    int values2[14] = {10,20,30,40,50,60,70,90,100,120,130,140,160,170};

```

```

    for (int i = 0; i < 14; i++) {
        root = insert(root, values[i]);
        root2 = insert(root2, values2[i]);
    }

```

```

    printf("Tree 1:\n");
    inorderTraversal(root); printf("\n");
    preorderTraversal(root); printf("\n");
    postorderTraversal(root); printf("\n");

```

```

    int level = find(root, 170, 0);
    printf("Level of 170 = %d\n\n", level);

```

```

    printf("Tree 2:\n");
    inorderTraversal(root2); printf("\n");
    preorderTraversal(root2); printf("\n");
    postorderTraversal(root2); printf("\n");

```



```

int level2 = find(root2, 170, 0);
printf("Level of 170 = %d\n", level2);

return 0;
}

```

10. implementation of Binary Search Tree with Traversals and Node Level Finding

```

#include <stdio.h>
#include <stdlib.h>

```

```

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

```

```

Node *newNode(int data) {
    Node *n = malloc(sizeof(Node));
    n->data = data;
    n->left = n->right = NULL;
    return n;
}

```

```

Node *insert(Node *root, int data) {
    if (!root) return newNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);
    return root;
}

```

```

Node *minNode(Node *root) {
    while (root->left)
        root = root->left;
    return root;
}

```

```

Node *deleteNode(Node *root, int data) {
    if (!root) return NULL;
    if (data < root->data)
        root->left = deleteNode(root->left, data);
    else if (data > root->data)
        root->right = deleteNode(root->right, data);
    else {
        if (!root->left) { Node *t = root->right; free(root); return t; }
        if (!root->right) { Node *t = root->left; free(root); return t; }
        Node *s = minNode(root->right);

```

```

    root->data = s->data;
    root->right = deleteNode(root->right, s->data);
}
return root;
}

void inorder(Node *root) {
    if (!root) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

int main(void) {
    int order[] = {10, 5, 21, 2, 7, 15, 24, 4, 6, 12, 17, 22, 29};
    Node *root = NULL;
    for (int i = 0; i < 13; i++)
        root = insert(root, order[i]);

    printf("Original: "); inorder(root); printf("\n");

    root = insert(root, 4);
    printf("After insert 4: "); inorder(root); printf("\n");

    root = deleteNode(root, 2);
    root = deleteNode(root, 7);
    root = deleteNode(root, 21);
    printf("After delete 2,7,21: "); inorder(root); printf("\n");

    return 0;
}

```

11. Implementation of Binary Search Tree with Insertion, Traversals, Search Count, and Deletion

```

#include <stdio.h>
#include <stdlib.h>

int count = 0;

struct node {
    int data;
    struct node* left;
    struct node* right;
};

struct node* createnode(int data) {
    struct node* temp = (struct node*)malloc(sizeof(struct node));
    temp->data = data;
}

```

```

    temp->left = NULL;
    temp->right = NULL;
    return temp;
}

struct node* insert(struct node *root, int data) {
    if (root == NULL)
        return createnode(data);
    else if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);
    return root;
}

struct node* find(struct node* root, int data) {
    if (root == NULL) return NULL;
    count = count + 1;
    if (root->data == data)
        return root;
    if (data < root->data)
        return find(root->left, data);
    else
        return find(root->right, data);
}

void inorderTraversal(struct node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

void preorderTraversal(struct node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}

void postorderTraversal(struct node* root) {
    if (root != NULL) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        printf("%d ", root->data);
    }
}

struct node* deleteNode(struct node* root, int data) {

```

```
if (root == NULL) return root;
```

```
if (data < root->data)
```

```
    root->left = deleteNode(root->left, data);
```

```
else if (data > root->data)
```

```
    root->right = deleteNode(root->right, data);
```

```
else {
```

```
    if (root->left == NULL && root->right == NULL) {
```

```
        free(root);
```

```
        return NULL;
```

```
    } else if (root->left == NULL) {
```

```
        struct node* temp = root->right;
```

```
        free(root);
```

```
        return temp;
```

```
    } else if (root->right == NULL) {
```

```
        struct node* temp = root->left;
```

```
        free(root);
```

```
        return temp;
```

```
    } else {
```

```
        struct node* temp = root->right;
```

```
        while (temp->left != NULL) {
```

```
            temp = temp->left;
```

```
        }
```

```
        root->data = temp->data;
```

```
        root->right = deleteNode(root->right, temp->data);
```

```
    }
```

```
}
```

```
return root;
```

```
}
```

```
int main() {
```

```
    struct node* root1 = NULL;
```

```
    root1 = insert(root1, 90);
```

```
    root1 = insert(root1, 60);
```

```
    root1 = insert(root1, 140);
```

```
    root1 = insert(root1, 40);
```

```
    root1 = insert(root1, 70);
```

```
    root1 = insert(root1, 120);
```

```
    root1 = insert(root1, 170);
```

```
    root1 = insert(root1, 20);
```

```
    root1 = insert(root1, 50);
```

```
    root1 = insert(root1, 10);
```

```
    root1 = insert(root1, 30);
```

```
    root1 = insert(root1, 100);
```

```
    root1 = insert(root1, 130);
```

```
    root1 = insert(root1, 160);
```

```
    printf("Preorder Traversal: ");
```

```
    preorderTraversal(root1);
```

```
    printf("\nInorder Traversal: ");
```

```
    inorderTraversal(root1);
```

```
printf("\nPostorder Traversal: ");  
postorderTraversal(root1);
```

```
count = 0;  
find(root1, 170);  
printf("\nNodes visited for 170: %d", count);
```

```
struct node* root2 = NULL;  
root2 = insert(root2, 10);  
root2 = insert(root2, 20);  
root2 = insert(root2, 30);  
root2 = insert(root2, 40);  
root2 = insert(root2, 50);  
root2 = insert(root2, 60);  
root2 = insert(root2, 70);  
root2 = insert(root2, 90);  
root2 = insert(root2, 100);  
root2 = insert(root2, 120);  
root2 = insert(root2, 130);  
root2 = insert(root2, 140);  
root2 = insert(root2, 160);  
root2 = insert(root2, 170);
```

```
printf("\n\nTree 2 - Preorder: ");  
preorderTraversal(root2);  
printf("\nTree 2 - Inorder: ");  
inorderTraversal(root2);  
printf("\nTree 2 - Postorder: ");  
postorderTraversal(root2);
```

```
count = 0;  
find(root2, 170);  
printf("\nNodes visited for 170 in Tree 2: %d\n", count);
```

```
return 0;
```

```
}
```

12. Height of binary tree

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

// create node
struct node* createnode(int data) {
    struct node* temp = (struct node*)malloc(sizeof(struct node));
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}

// function to find height
int height(struct node* root) {
    if (root == NULL)
        return 0;

    int left = height(root->left);
    int right = height(root->right);

    return (left > right ? left : right) + 1;
}

int main() {
    struct node* root = NULL;

    // manually creating tree
    root = createnode(10);
    root->left = createnode(20);
    root->right = createnode(30);
    root->left->left = createnode(40);
    root->left->right = createnode(50);

    int h = height(root);
    printf("Height of tree = %d\n", h);

    return 0;
}
```

13. Level of binary tree

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

// create node
struct node* createnode(int data) {
    struct node* temp = (struct node*)malloc(sizeof(struct node));
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}

// function to find level
int findLevel(struct node* root, int key, int level) {
    if (root == NULL)
        return -1;

    if (root->data == key)
        return level;

    int left = findLevel(root->left, key, level + 1);
    if (left != -1)
        return left;

    return findLevel(root->right, key, level + 1);
}

int main() {
    struct node* root = NULL;

    // creating tree
    root = createnode(1);
    root->left = createnode(2);
    root->right = createnode(3);
    root->left->left = createnode(4);
    root->left->right = createnode(5);

    int level = findLevel(root, 5, 1); // start level from 1
    printf("Level of node = %d\n", level);

    return 0;
}
```

14.construct avl tree

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
    int height;
};

// height
int h(struct node* n) {
    return n ? n->height : 0;
}

// max
int max(int a, int b) {
    return (a > b) ? a : b;
}

// create node
struct node* newnode(int x) {
    struct node* t = (struct node*)malloc(sizeof(struct node));
    t->data = x;
    t->left = t->right = NULL;
    t->height = 1;
    return t;
}

// right rotation
struct node* rightRotate(struct node* y) {
    struct node* x = y->left;
    y->left = x->right;
    x->right = y;

    y->height = max(h(y->left), h(y->right)) + 1;
    x->height = max(h(x->left), h(x->right)) + 1;

    return x;
}

// left rotation
struct node* leftRotate(struct node* x) {
    struct node* y = x->right;
    x->right = y->left;
    y->left = x;

    x->height = max(h(x->left), h(x->right)) + 1;
    y->height = max(h(y->left), h(y->right)) + 1;

    return y;
}
```



```

}

// balance factor
int bf(struct node* n) {
    return n ? h(n->left) - h(n->right) : 0;
}

// insert
struct node* insert(struct node* root, int x) {
    if (!root) return newnode(x);

    if (x < root->data)
        root->left = insert(root->left, x);
    else if (x > root->data)
        root->right = insert(root->right, x);

    root->height = 1 + max(h(root->left), h(root->right));

    int b = bf(root);

    // rotations
    if (b > 1 && x < root->left->data)
        return rightRotate(root);

    if (b < -1 && x > root->right->data)
        return leftRotate(root);

    if (b > 1 && x > root->left->data) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }

    if (b < -1 && x < root->right->data) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

    return root;
}

// inorder
void inorder(struct node* root) {
    if (root) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct node* root = NULL;

```

```

root = insert(root, 10);
root = insert(root, 20);
root = insert(root, 30);
root = insert(root, 40);

printf("Inorder: ");
inorder(root);

return 0;
}

```

15. Multi-way search tree -3 way

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data1, data2;
    struct node *left, *middle, *right;
};

// create node
struct node* create(int x) {
    struct node* t = (struct node*)malloc(sizeof(struct node));
    t->data1 = x;
    t->data2 = -1; // empty
    t->left = t->middle = t->right = NULL;
    return t;
}

// insert (basic logic, not full balanced)
struct node* insert(struct node* root, int x) {
    if (root == NULL)
        return create(x);

    if (root->data2 == -1) {
        if (x < root->data1) {
            root->data2 = root->data1;
            root->data1 = x;
        } else {
            root->data2 = x;
        }
        return root;
    }

    if (x < root->data1)
        root->left = insert(root->left, x);
    else if (x > root->data2)
        root->right = insert(root->right, x);
    else

```

```

        root->middle = insert(root->middle, x);

    return root;
}

// traversal
void inorder(struct node* root) {
    if (root) {
        inorder(root->left);
        printf("%d ", root->data1);
        inorder(root->middle);
        if (root->data2 != -1)
            printf("%d ", root->data2);
        inorder(root->right);
    }
}

int main() {
    struct node* root = NULL;

    root = insert(root, 30);
    root = insert(root, 10);
    root = insert(root, 50);
    root = insert(root, 20);

    inorder(root);
    return 0;
}

```

16. Multiway Search Tree -4 way

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data[3]; // max 3 keys
    int count;
    struct node* child[4];
};

// create node
struct node* create(int x) {
    struct node* t = (struct node*)malloc(sizeof(struct node));
    t->data[0] = x;
    t->count = 1;
    for (int i = 0; i < 4; i++)
        t->child[i] = NULL;
    return t;
}

```

```

// insert (very basic, no splitting)
struct node* insert(struct node* root, int x) {
    if (root == NULL)
        return create(x);

    if (root->count < 3) {
        root->data[root->count++] = x;
        return root;
    }

    if (x < root->data[0])
        root->child[0] = insert(root->child[0], x);
    else if (x < root->data[1])
        root->child[1] = insert(root->child[1], x);
    else if (x < root->data[2])
        root->child[2] = insert(root->child[2], x);
    else
        root->child[3] = insert(root->child[3], x);

    return root;
}

// traversal
void inorder(struct node* root) {
    if (root) {
        for (int i = 0; i < root->count; i++) {
            inorder(root->child[i]);
            printf("%d ", root->data[i]);
        }
        inorder(root->child[root->count]);
    }
}

int main() {
    struct node* root = NULL;

    root = insert(root, 10);
    root = insert(root, 20);
    root = insert(root, 30);
    root = insert(root, 40);

    inorder(root);
    return 0;
}

```

17. Binary search tree

```
#include <stdio.h>
#include <stdlib.h>
```

NODE STRUCTURE

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

CREATE NODE

```
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

INSERT

```
struct Node* insert(struct Node* root, int value) {
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    else
        printf("Duplicate value %d not inserted.\n", value);

    return root;
}
```

SEARCH

```
struct Node* search(struct Node* root, int value) {
    if (root == NULL) {
        printf("%d not found in tree.\n", value);
        return NULL;
    }

    if (value == root->data) {
        printf("Found %d in the tree.\n", value);
        return root;
    }
}
```

```

}
else if (value < root->data)
    return search(root->left, value);
else
    return search(root->right, value);
}

```

FIND MINIMUM

```

// Used in deletion to find in-order successor
struct Node* findMin(struct Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

```

FIND MAXIMUM

```

struct Node* findMax(struct Node* root) {
    while (root->right != NULL)
        root = root->right;
    return root;
}

```

DELETE

```

struct Node* delete(struct Node* root, int value) {
    if (root == NULL) {
        printf("%d not found, cannot delete.\n", value);
        return NULL;
    }

    if (value < root->data)
        root->left = delete(root->left, value);

    else if (value > root->data)
        root->right = delete(root->right, value);

    else {
        // Case 1: Leaf node
        if (root->left == NULL && root->right == NULL) {
            printf("Deleted leaf node %d\n", value);
            free(root);
            return NULL;
        }

        // Case 2: One child (right only)
        else if (root->left == NULL) {
            struct Node* temp = root->right;
            printf("Deleted node %d (had right child)\n", value);
            free(root);

```

```

        return temp;
    }

    // Case 2: One child (left only)
    else if (root->right == NULL) {
        struct Node* temp = root->left;
        printf("Deleted node %d (had left child)\n", value);
        free(root);
        return temp;
    }

    // Case 3: Two children
    // Find in-order successor (smallest in right subtree)
    else {
        struct Node* successor = findMin(root->right);
        printf("Deleted node %d, replaced with successor %d\n",
            root->data, successor->data);
        root->data = successor->data;
        root->right = delete(root->right, successor->data);
    }
}
return root;
}

```

TRAVERSALS

// In-Order → Left, Root, Right (gives sorted output)

```

void inOrder(struct Node* root) {
    if (root == NULL) return;
    inOrder(root->left);
    printf("%d ", root->data);
    inOrder(root->right);
}

```

// Pre-Order → Root, Left, Right

```

void preOrder(struct Node* root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preOrder(root->left);
    preOrder(root->right);
}

```

// Post-Order → Left, Right, Root

```

void postOrder(struct Node* root) {
    if (root == NULL) return;
    postOrder(root->left);
    postOrder(root->right);
    printf("%d ", root->data);
}

```

TREE HEIGHT

```
int height(struct Node* root) {
    if (root == NULL) return 0;

    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    return 1 + (leftHeight > rightHeight ? leftHeight : rightHeight);
}
```

COUNT NODES

```
int countNodes(struct Node* root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

FREE TREE

```
void freeTree(struct Node* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}
```

MAIN

```
int main() {
    struct Node* root = NULL;

    // --- Insert ---
    int values[] = {50, 30, 70, 20, 40, 60, 80, 10, 25};
    int size = sizeof(values) / sizeof(values[0]);

    for (int i = 0; i < size; i++) {
        root = insert(root, values[i]);
        printf("Inserted: %d\n", values[i]);
    }

    // --- Traversals ---
    printf("In-Order (Sorted) : "); inOrder(root); printf("\n");
    printf("Pre-Order (Root->L->R): "); preOrder(root); printf("\n");
    printf("Post-Order (L->R->Root): "); postOrder(root); printf("\n");

    // --- Min / Max ---
    printf("Minimum value: %d\n", findMin(root)->data);
}
```



```
printf("Maximum value: %d\n", findMax(root)->data);

// --- Height & Count ---
printf("Tree Height : %d\n", height(root));
printf("Total Nodes : %d\n", countNodes(root));

// --- Search ---
search(root, 25);
search(root, 99);

// --- Delete ---
root = delete(root, 20); // leaf node
root = delete(root, 30); // one child
root = delete(root, 50); // two children (root)
root = delete(root, 99); // not found

printf("\nIn-Order after deletions: ");
inOrder(root);
printf("\n");

// --- Free Memory ---
freeTree(root);

return 0;
}
```